



The Masked Signal

A field guide to digital privacy—
Linux, GrapheneOS, Silent.Link, Mullvad, Tor, and Nostr

TL;DR

Privacy is a stack, not a purchase. No single app or setting protects you; the tools below cover different layers of exposure, and each only does its job if the layer beneath it is sound.

- **Threat model first.** Decide what you are protecting, from whom, and how much friction you are willing to accept — before buying or installing anything.
- **Linux** removes the always-on telemetry baked into mainstream desktop operating systems and gives you a system whose behaviour you (or the community around you) can actually inspect.
- **GrapheneOS** does the same job for a phone: a hardened, de-Google'd Android with no default connection to Google's servers, running on Pixel hardware with a verifiable boot chain.
- **Silent.Link** solves the leftover phone-number problem: a Bitcoin-and-Monero-only eSIM that asks for no name, no email, and no card, decoupling your number from your legal identity at the point of sale.
- **Mullvad** hides your network traffic from your ISP and local observers and, unusually for a VPN, does not ask who you are — no email, no name, just a number.
- **Tor** goes further: it splits your traffic across three independently-operated relays so that no single party, including Tor itself, can see both who you are and what you are visiting.
- **Nostr** replaces the username-and-password account with a cryptographic keypair, so your identity and your social graph belong to you rather than to whichever platform is hosting them this year.

This is a practical starting point, not a complete operational-security manual. It assumes a reasonably capable adult reader with an ordinary laptop and phone, and it favours tools that are free, open-source, and independently audited where possible. Nothing here helps you break the law; it helps you keep the state, the advertiser, and the data broker on the same footing — guessing, not knowing.

Why a Rat in a Mask

This paper sits oddly beside the ones on monetary history, and only at first glance. The earlier essays in this imprint have spent a great deal of ink on self-custody of money — on why holding your own keys to your own coins matters more than the interest rate printed on a bank statement. Privacy is the same argument applied one layer up. A Bitcoin holder who signs every transaction from a phone leaking its owner's identity to six advertising networks has only solved half the problem. Sovereignty over money without sovereignty over the wire that money travels down is a fairly thin kind of sovereignty.

None of what follows is about hiding wrongdoing. The useful frame, borrowed from the privacy and civil-liberties world, is information asymmetry. A modern smartphone, unremarkable and unconfigured, reports its owner's location, contacts, app usage, and browsing habits to a handful of companies as a matter of routine, uncompensated and mostly unconsented-to in any meaningful sense. None of that is illegal. Almost none of it is even unusual. But asymmetric knowledge is power, and a citizen who cannot tell who is watching, or what they intend to do with what they see, is negotiating from a position of permanent disadvantage — with advertisers over price, with insurers over risk, and with the state over consequence.

The five tools covered here — Linux, GrapheneOS, Mullvad, Tor, and Nostr — are not a single product bundle and were not designed by the same people to work together. They address five distinct points of exposure: the operating system that runs your computer, the operating system that runs your phone, the network your traffic crosses, the anonymity of that traffic against a well-resourced observer, and the platform that hosts your online identity. Treat this as a map of the terrain, not a shopping list to complete in one afternoon.

Threat Modelling First

Before touching a single setting, it is worth five minutes with a pen. The Electronic Frontier Foundation's Surveillance Self-Defense project reduces this to five questions, and they are worth answering in order, honestly, for your own situation rather than a hypothetical one: What do you want to protect? Who do you want to protect it from? How bad are the consequences if you fail? How likely is that adversary to actually come after you? And how much inconvenience are you willing to tolerate to prevent it?

The answers matter because the tools below are not interchangeable, and over-engineering a threat model is its own failure mode — wasted time, unnecessary friction, and a strong temptation to abandon the whole project in frustration. A journalist protecting a source from a state intelligence service needs Tor, compartmentalised hardware, and considerable discipline. Someone who simply objects to being profiled by ad networks needs a fraction of that stack and will get most of the benefit from a hardened browser and a decent VPN.



FIG. 1 — The EFF's five-question threat-modelling framework, adapted from Surveillance Self-Defense

Two adversary classes are worth distinguishing early because the tools respond to them differently. A **passive, broad** adversary — ad-tech networks, data brokers, an ISP selling browsing metadata — is scooping up everything it can cheaply reach and correlating it later. A **targeted, resourced** adversary — a state security service, a determined stalker, a well-funded corporate investigator — is looking specifically for you and is willing to spend

real effort and money to find you. Mullvad alone is a strong answer to the first. It is a much weaker answer to the second, because a VPN provider is itself a single point of trust and, in some jurisdictions, a single point of legal pressure.

RULE OF THUMB

Match the tool to the adversary you actually have, not the scariest one you can imagine. Running Tor for everything, all the time, when your real concern is ad-network tracking, mostly just makes web pages load slowly.

The Operating System Layer

Everything else in this essay is built on sand if the device underneath it is compromised or simply reporting your activity home by default. This is the layer people skip, because it is the most disruptive to change, and it is also the layer that matters most: a leaking operating system defeats a VPN, a hardened browser, and a careful Nostr key all at once, because it can simply read them off the screen or out of memory.

Linux, on the Desktop

Windows and macOS are not neutral platforms. Both ship extensive telemetry by default, both are closed-source (so no outside party can verify what that telemetry actually contains), and both are commercial products whose vendors have, at various points, experimented with advertising, cloud-account requirements, and AI features that read local files. None of this makes either operating system unusable — most of the world runs on them without incident — but neither is a serious foundation for a privacy-conscious setup.

Linux is a family of operating systems built on an open-source kernel, with the source code of the whole stack available for inspection by anyone. That transparency does not guarantee security on its own, but it means claims about behaviour can actually be checked rather than taken on trust. For a newcomer, distribution choice matters less than people assume; almost any mainstream, actively-maintained distribution is a substantial improvement in visibility and control over a stock consumer OS.

- **Linux Mint** and **Fedora** are the two most commonly recommended starting points for someone migrating from Windows: both have mature hardware support, large communities, and interfaces that do not require a command line to use day-to-day.
- **Debian** sits a step further toward the stability-and-minimalism end of the spectrum and is a common choice once a user is comfortable with the basics.
- **Qubes OS**, at the far end, isolates every application in its own virtual machine and is worth knowing about even if it is overkill for most people — it is what a targeted, high-consequence threat model actually calls for.

Whichever distribution is chosen, four habits do most of the work:

- 1. Full-disk encryption** (LUKS on most distributions) at install time, so a lost or seized laptop is unreadable without the passphrase.
- 2. A firewall** — `ufw` is the simplest front-end — set to deny incoming connections by default.
- 3. Software from the distribution's own repositories** wherever possible, rather than downloaded binaries of uncertain provenance.
- 4. Timely updates.** An unpatched Linux box is not meaningfully safer than an unpatched Windows one; the security model depends on the patches actually being applied.

CAVEAT

Desktop Linux does not, by itself, anonymise your network traffic or hide your identity from the sites you visit. It removes one layer of exposure — the operating-system vendor — and leaves the network and application layers to the tools covered in the chapters that follow.

GrapheneOS, on the Phone

The phone is the harder problem, because modern mobile operating systems are built around an identity system — a Google or Apple account — that the whole platform assumes you want. GrapheneOS is a hardened fork of the Android Open Source Project that removes that assumption. It ships with no Google apps and no default connection to Google's servers; Google Play services, if wanted at all, run afterwards as a sandboxed, permission-limited app rather than as a privileged part of the operating system.

Beyond de-Googling, GrapheneOS carries a substantial list of security hardening work that stock Android does not: a hardened memory allocator designed to make memory-corruption exploits harder to weaponise, stricter application sandboxing, a re-worked permission system with more granular network and sensor controls, and full verified boot — meaning the phone will refuse to boot software that has been tampered with, and will tell you so.

The practical catch is hardware. GrapheneOS currently runs only on Google's own Pixel phones, from the Pixel 6 series onward, because Pixels are, at present, the only mainstream devices that meet its requirements for an unlockable-and-relockable bootloader with a verifiable boot chain. The project has said it is working with a major Android manufacturer to bring official support to Qualcomm Snapdragon-based devices, expected

sometime around 2026–2027, which would end the single-vendor dependency — worth checking on before you buy, since it may have landed by the time you read this.

- 1.** Buy a supported Pixel, ideally new or from a trusted source, with an unlockable bootloader and no carrier lock.
- 2.** Install via the official GrapheneOS web installer, which talks to the phone directly over USB from a Chromium-based browser — no separate computer setup required.
- 3.** Set up device encryption and a strong lock-screen passphrase during initial configuration; this is the actual key that protects data at rest.
- 4.** Decide, app by app, whether you need Sandboxed Google Play at all. Many people find they do not.
- 5.** Use GrapheneOS's separate user profiles to keep, say, a "public" profile with the apps you cannot avoid, and a more locked-down profile for anything sensitive.

NOTE

A hardened phone with your real name, real payment cards, and real social accounts logged into it is still a phone that knows exactly who you are. GrapheneOS raises the cost of a technical compromise; it does not by itself anonymise you, and it does not touch the SIM card sitting in the phone — which, in most countries, is tied to your identity by law. That is next.

The Phone Number Problem — Silent.Link

A hardened operating system solves nothing if the SIM card underneath it was bought with a passport photo and a credit card. Mobile-network operators in most jurisdictions are required by law to record who holds a given SIM, and that record links a phone number — and everything sent to or from it — directly to a legal identity. This is a separate problem from the operating-system and network-traffic layers already covered, and neither GrapheneOS nor Mullvad touches it: the carrier's KYC file exists regardless of what software the phone is running.

Silent.Link is a Bitcoin-and-Monero-only eSIM provider built specifically around this gap. It sells prepaid, pay-as-you-go mobile data and SMS with no sign-up information at all — no name, no email address, no card on file — because the only accepted payment rails are Bitcoin and Monero, which do not require one. Its own site is reachable as a Tor onion service for the sign-up itself. An eSIM profile is issued against nothing but a payment; there is no account in the conventional sense to compromise, subpoena, or leak.

HOW IT DIFFERS FROM AN ORDINARY PREPAID SIM

Two design choices do most of the work. First, the eSIM is always used in **roaming mode** — it is registered to a home network abroad and roams onto whichever local carrier is available, wherever the phone actually is. A roaming connection hands the visited network less information than a native subscriber connection would, and closes off a class of attacks — such as SIM-toolkit abuse, where a local network operator with your real subscriber details can push silent commands to the SIM itself — that depend on the local carrier already knowing who you are. Second, the balance never expires and there are no recurring charges, so there is no monthly billing relationship to maintain, and by extension no ongoing paper trail of payments tied to a renewing subscription.

WHAT IT SOLVES	WHAT IT DOES NOT SOLVE
No KYC record linking a phone number to a legal identity at the provider	The phone's IMEI (hardware serial) is still visible to every network it connects to
Local carrier sees a roaming subscriber, not your real subscriber profile	Cell-tower triangulation of the device's location remains possible any time the modem is on

No recurring billing relationship or expiring subscription to maintain	SMS and calls over the mobile network are not end- to-end encrypted
---	--

CAVEAT

An eSIM, however it was purchased, is still a device on a network built to track devices. It reveals hardware identifiers to every tower it touches, and nothing about payment anonymity changes that. Pay with Monero rather than Bitcoin if the payment trail itself is part of the threat model — a transparent Bitcoin payment can, in principle, be traced back through the exchange or Lightning node that funded it, where Monero's design makes that considerably harder. Treat Silent.Link as removing the identity requirement at the point of sale, not as making the phone invisible to the mobile network it has to join to work at all.

In practice this makes the most sense paired with the previous chapter's hardware: a GrapheneOS phone carrying a Silent.Link eSIM gives a number that was never tied to a name at either the operating-system or the carrier level, useful for SMS verification codes, a travel data connection, or a secondary line kept separate from a primary, identified number. It is not, on its own, an anonymous phone — nothing running on a cellular network fully is — but it removes the single weakest link in an otherwise hardened setup: the paperwork trail a SIM card ordinarily leaves at the point of purchase.

The Network Layer — Mullvad and Tor

A hardened device still has to talk to the outside world, and that conversation passes through an internet service provider who can see, at minimum, which servers you are contacting and when, even if the content itself is encrypted. This chapter covers two different answers to that exposure, and the difference between them matters more than most marketing material suggests.

Mullvad: Hiding Traffic From the Local Observer

Mullvad is a VPN provider, and what sets it apart from most of the crowded VPN market is less its technology — WireGuard, a fast and well-reviewed modern protocol, is now fairly standard — than its approach to identity. Signing up requires no email address, no name, and no payment method that identifies you if you choose to pay in cash sent by post; Mullvad instead issues a numeric account number, and that number is the entirety of what it knows about you. The company has also submitted to independent, published audits of its no-logging claims.

What a VPN actually does is straightforward: it encrypts traffic between your device and the VPN server, so your ISP sees only an encrypted tunnel to Mullvad rather than a list of sites you visited, and the destination sites see Mullvad's IP address rather than yours. That is a real and useful improvement against a local network observer, a nosy ISP, or an untrusted coffee-shop Wi-Fi network.

The Tor Project: Anonymity Against a Stronger Observer

Tor solves a different, harder problem. A VPN provider is itself a single party who can, in principle, see both who you are (your real IP) and what you are doing (your destination) at the same time — you have simply moved your trust from the ISP to the VPN operator. Tor's onion-routing design is built specifically to avoid that single point of knowledge, by routing traffic through three independently-run relays in sequence, each of which only knows the step immediately before and after it.

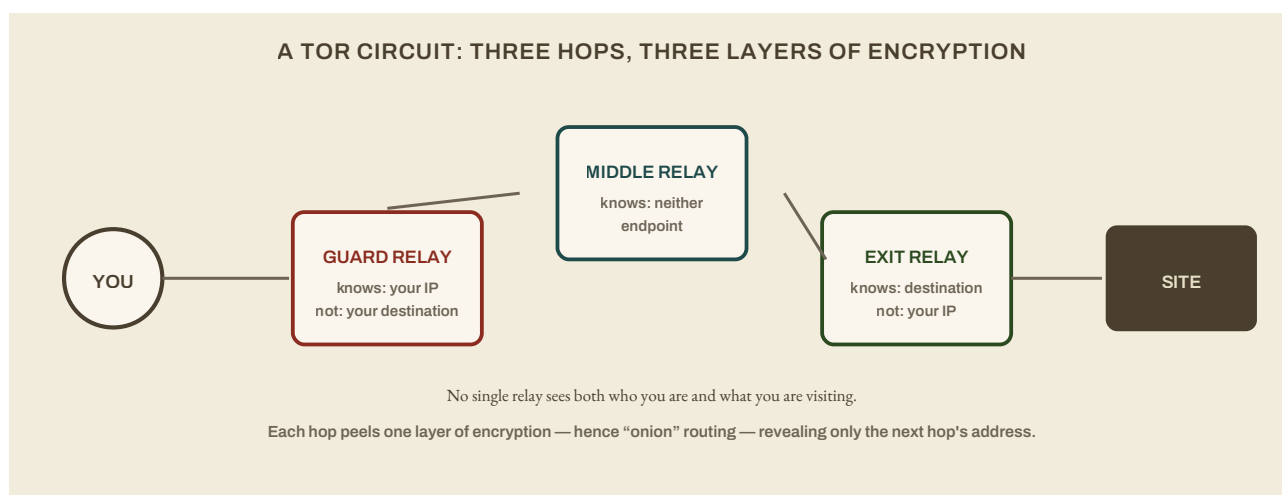


FIG. 2 — A simplified Tor circuit. The real network has thousands of volunteer-run relays; a fresh three are chosen for each circuit.

The **guard relay** knows your real IP address but not your destination. The **exit relay** knows the destination but not you. The **middle relay** knows neither. Compromising this design requires either controlling a large fraction of the relay network at once, or watching traffic enter and leave the network simultaneously and statistically correlating timing patterns — a capability generally attributed only to the largest state-level observers, and one Tor's design cannot fully defend against no matter how it is configured.

The Tor Browser, the recommended way to use the network, is hardened specifically against browser fingerprinting: it standardises window size, disables or restricts APIs commonly used to fingerprint devices, and isolates cookies and cache per-site so that tracking across sites is far harder than in an ordinary browser. Tor also enables access to **.onion services** — sites reachable only within the Tor network, used by newsrooms, whistleblower drop-boxes, and privacy-focused services that want to offer a route with no exit relay at all, and therefore no point at which traffic touches the ordinary internet in the clear.

PROPERTY	MULLVAD VPN	TOR
Hides traffic from	Your ISP and local network	Your ISP and any single relay
Who could still identify you	Mullvad itself, in theory, or the destination site if you log in	A global adversary correlating traffic timing at both ends
Speed	Near-native; suitable for streaming, calls	Noticeably slower; three hops plus volunteer bandwidth

Trust model	Single provider, audited no-logs policy	Decentralised; no single relay operator trusted
Best suited to	Everyday browsing, hiding usage from ISPs, geo-restricted access	Higher-stakes anonymity: sources, activists, sensitive research

The two are sometimes combined, but the ordering matters and is genuinely debated within the privacy community rather than a settled matter of technical consensus. Connecting to Tor through a VPN hides the fact that you are using Tor from your ISP, but adds the VPN provider as another party who can see your real IP; connecting to a VPN through Tor's exit node protects against a malicious exit relay but is more complex to set up correctly. For most people in most threat models, the simplest and least error-prone approach is to use each tool separately for the job it is best at: Mullvad for daily browsing, Tor Browser on its own when the situation actually calls for it.

The Identity Layer — Nostr

Every layer so far has been about hiding traffic. Nostr addresses a different exposure: the fact that almost all online identity today is rented, not owned. A Twitter or Instagram account is a row in a database the platform controls; the platform can suspend it, sell access to it, change the rules it operates under, or simply shut down, and the account — along with the audience and history attached to it — disappears with no recourse.

PROTOCOL, NOT PLATFORM

Nostr — "Notes and Other Stuff Transmitted by Relays" — is not an app or a company. It is an open protocol: a small, simple specification for signed messages, published as a set of documents called NIPs (Nostr Implementation Possibilities) maintained collaboratively on GitHub. Anyone can write a client that speaks the protocol, and anyone can run a relay — a server that stores and forwards messages — without asking permission from a central authority, because there isn't one.

Identity on Nostr is a cryptographic keypair, not a username and password. A private key (`nsec`) signs everything you publish; the corresponding public key (`npub`) is your identity, verifiable by anyone without needing to trust a server's claim about who posted what. This is the same elliptic-curve mechanism underpinning Bitcoin transaction signatures, applied to social content instead of monetary value — a reader of the earlier essays in this series will recognise the pattern immediately.

ACCOUNT MODEL VS. PROTOCOL MODEL

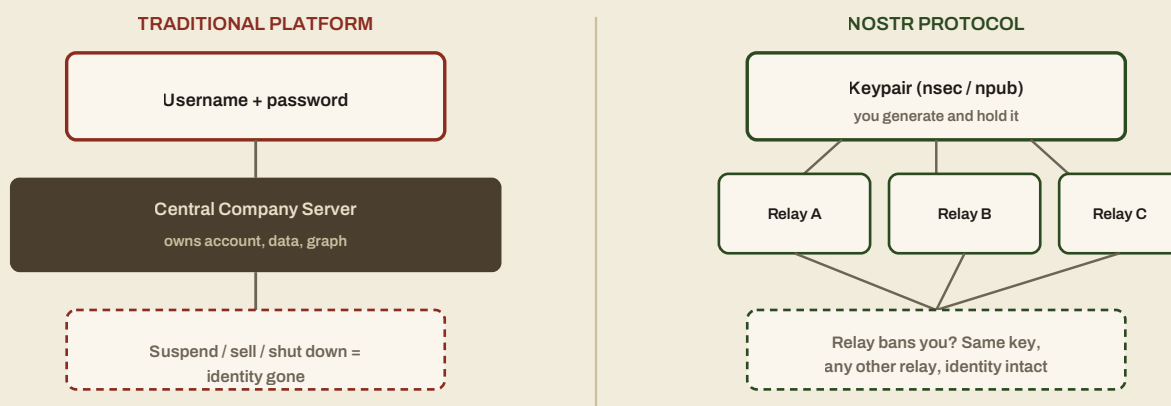


FIG. 3 — Nostr moves identity out of any single company's database and into a keypair the user holds directly.

Because identity is portable, censorship on Nostr works differently than on a conventional platform. A relay can refuse to store or forward your messages — that is entirely within its rights, since it is a voluntarily-run piece of infrastructure, not a public utility — but doing so does not delete your account, your followers, or your history, because none of those things live on that relay alone. Publish the same signed note to a different relay and your identity, and everyone following your public key, follows straight through. Deplatforming a Nostr identity, rather than a single relay's copy of it, is considerably harder than deplatforming a conventional social-media account.

THE TRADE-OFF

There is no password reset. If the private key is lost, the identity is lost, permanently and with no customer-support appeal process, exactly as with a Bitcoin seed phrase. Back it up the same way: written down, offline, ideally split or stored in a way that survives a single point of failure — the seed-phrase discipline covered in an earlier essay in this series applies here without modification.

Getting started is a matter of choosing a client — there are dozens, for web, desktop, and mobile, since anyone can build one against the open specification — generating a keypair, and connecting to a handful of relays (most clients ship with reasonable defaults). Many users also register a human-readable NIP-05 identifier, which maps a familiar-looking address to the underlying public key purely for convenience; it is a courtesy for other humans, not a requirement of the protocol, and it can be pointed at a domain you control.

Putting It Together

Stacked on top of each other, these tools form a rough hierarchy, each layer only as trustworthy as the one beneath it. A Nostr identity is only as safe as the device signing its events; a Tor circuit is only as anonymous as the operating system routing traffic into it; a Silent.Link eSIM is only as useful as the phone it sits in; and none of it matters if the hardware itself is compromised.

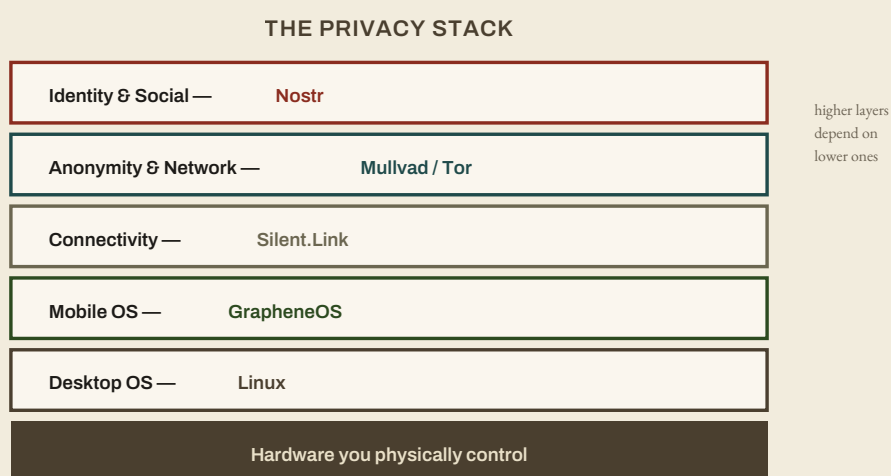


FIG. 4 — Each layer only holds if the layer below it does. Build from the bottom.

For someone starting from a completely stock setup, the order that produces the least disruption per unit of privacy gained tends to run roughly as follows. It is a starting suggestion, not a rulebook — revisit the five questions from Chapter One and adjust freely.

- 1. Harden the browser and add a VPN.** Switch to a privacy-respecting browser, and subscribe to Mullvad. This is the lowest-effort, highest-return step and takes an afternoon.
- 2. Migrate the desktop to Linux,** starting with a secondary or old machine if a full switch feels risky. Most everyday software — browsers, office suites, media players — has a direct or near-direct Linux equivalent.
- 3. Move to a GrapheneOS phone** when the current phone is due for replacement anyway, rather than as an urgent separate purchase.

4. **Pick up a Silent.Link eSIM** for that phone once it is running, so the number itself carries no KYC record from day one.
5. **Install Tor Browser** for the specific activities that warrant it — sensitive research, whistleblowing-adjacent communication, anything where the adversary might plausibly be a well-resourced one — rather than as a full-time daily browser.
6. **Set up a Nostr identity** once the underlying device and network are already in reasonable shape, so the key is generated and stored on something worth trusting.

Privacy is a practice, not a purchase. The tools remove friction from doing the sensible thing; they do not do the sensible thing on your behalf.

A final, unglamorous point: every tool in this stack is legal to use in the United Kingdom and in the great majority of jurisdictions worldwide, exactly as a VPN, a Linux laptop, or an encrypted messaging app is legal. A minority of states restrict or ban some of them outright, so it is worth a moment's check of local law before travelling with any of this configured on a device. None of it requires bad intent to justify; wanting the ad-tech industry and the local Wi-Fi network to know a little less about you is, on its own, a perfectly sufficient reason.

Summary

The Whole Stack, One Habit

- Privacy tooling works as a **stack**: operating system, connectivity, network, and identity each need attention, and weakness in one undermines the others.
- **Threat-model before tooling**. The right setup depends entirely on who you are actually worried about and what you are willing to trade for protection against them.
- **Linux and GrapheneOS** address the operating-system layer on desktop and mobile respectively, removing default telemetry and giving you a verifiable, inspectable foundation.
- **Silent.Link** closes the SIM-card gap a hardened phone leaves open, issuing a number with no KYC record behind it.
- **Mullvad** hides traffic from local observers with minimal friction; **Tor** defends against a stronger, better-resourced adversary at the cost of speed and convenience — know which job you actually need done.
- **Nostr** moves identity off a rented platform and onto a keypair you hold yourself, the same self-custody logic already familiar from Bitcoin, applied to who you are online rather than what you own.
- None of this is about hiding wrongdoing. It is about not negotiating from a position of permanent, unpaid, and largely invisible disadvantage.

FURTHER READING

References

1. **Electronic Frontier Foundation** — Surveillance Self-Defense: Your Security Plan, ssd.eff.org. The five-question threat-modelling framework used in Chapter One.
2. **GrapheneOS Project** — official documentation and FAQ, grapheneos.org. Device support, installation, and security architecture.
3. **Silent.Link** — service FAQ and pricing, silent.link (also reachable as a Tor onion service). KYC-free eSIM data and SMS, Bitcoin and Monero payment only.
4. **Mullvad VPN** — "No-logging" policy and published third-party audit reports, mullvad.net.
5. **The Tor Project** — Tor: Overview and design documentation, torproject.org. Onion-routing mechanics and Tor Browser hardening.
6. **Nostr Protocol** — NIPs (Nostr Implementation Possibilities) specification repository, github.com/nostr-protocol/nips.
7. **Privacy Guides** — community-maintained, non-commercial tool recommendations and threat-model explainers, privacyguides.org.
8. See also, in this series: Just How Secure Is a 24 Word BIP39 Bitcoin Seed Phrase?, for the key-backup discipline referenced in Chapter Five.

The Ratty2Tails Papers — The Machine Series, No. III. Published under the Ratty2Tails imprint. This essay reflects the tools and their capabilities as understood at the time of writing; software, supported hardware, and legal status of these tools can change, and readers should confirm current details against the primary sources above before relying on them.